XM · SESIÓN 1 · CLASES 13 Y 14

Soluciones de Analítica con Databricks

Bundles, ambientes y CI/CD: el mismo código, de dev a prod, por un camino que deja rastro

Docente: Manuela Larrea Gómez

Agenda de la sesión 1: clases 13 y 14

3 h 30 · del job creado por API a un producto que se promueve solo, con pruebas y una aprobación

10	MIN	**Apertura**	Lo que dejó la clase 12: el job por API y el chequeo que se excluía con la rama
30	MIN	**Clase 13 · Bundles y ambientes**	databricks.yml, recursos, variables, modos, qué cambia entre ambientes, Free Edition vs. Azure
50	MIN	**Lab 13**	Deploy a dev (y el que falla), target qa, preparar_ambiente, qa reproduce a dev
10	MIN	**Receso**	
25	MIN	**Clase 14 · CI/CD y promoción**	CI y CD, autenticación, compuertas, prueba de integración, promover código o modelo
45	MIN	**Lab 14**	Fork con secrets y environments · TODO deploy-prod · PR rojo, PR verde, push rechazado, tag
10	MIN	**Receso**	Mientras corre deploy-qa (unos 8 minutos)
20	MIN	**Aprobación y auditoría**	Approve en GitHub · qué versión corre en cada ambiente, con qué identidad · promover modelos
10	MIN	**Cierre**	Tags s13 y s14 · así sería en Azure · qué traer a la sesión 2

Lo que dejó la clase 12: un job que funciona, pero no es un producto

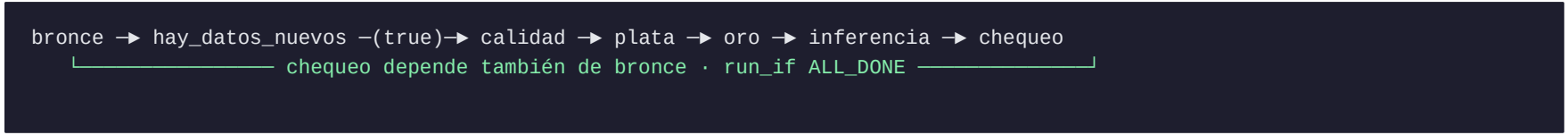
Y una lección del ensayo: un control solo sirve si alguien comprobó que se dispara

El job se creó desde un notebook

jobs/create o jobs/reset con un diccionario en Python. Corre la copia de tu Git folder, como tú, en tu esquema. Si alguien cambia un timeout en la interfaz, el repositorio ya no describe lo que corre. Y no hay una forma repetible de llevar exactamente lo mismo a otro ambiente.

El chequeo que se excluía con la rama

Primera versión: chequeo dependía solo de inferencia, con ALL_DONE. Sin archivo nuevo, la rama se excluyó y el chequeo con ella: Databricks excluye una tarea cuando todas sus dependencias están excluidas. El job terminó en verde. Corrección: depender también de bronce, y un test que lo exige.



Hoy: ese mismo job, declarado en Git, desplegado igual en dev, qa y prod, y promovido por un pipeline con pruebas.

Un bundle cambia la fuente de verdad: del workspace a Git

Declarative Automation Bundles (hasta marzo de 2026, Databricks Asset Bundles): código + recursos + ambientes

Pregunta	Job creado por API o en la UI	Job declarado en un bundle
¿Dónde está la verdad?	En el workspace: lo que haya quedado	En Git: el workspace es el resultado de un deploy
¿Cómo llego a otro ambiente?	Recrear a mano, o copiar el JSON	databricks bundle deploy -t qa
¿Quién revisa un cambio?	Nadie: se edita en vivo	Un PR, con pruebas, como cualquier código
¿Qué código corre?	La copia editable de un notebook	Los archivos sincronizados de un commit
¿Y si edito en la UI?	Se queda así hasta que alguien note	En modo producción, bloqueado; el siguiente deploy lo sobrescribe

Estado del producto en 2026: renombrado a Declarative Automation Bundles (16-mar-2026) · despliegue desde el workspace, GA desde oct-2025 · motor de despliegue directo, sin Terraform por debajo, GA en jun-2026 (CLI 1.3.0) · recursos también en Python.

Fuente: Databricks, notas de versión de Declarative Automation Bundles (2025–2026)

Anatomía de databricks.yml: qué es, qué corre, a dónde va

Un solo archivo en la raíz del repo; los recursos en resources/*.yml

databricks.yml (resumido)

```yaml
bundle:
  name: xm-demanda
include:
  - resources/*.yml
sync:
  exclude: [clases/**, data/raw/**, docs/**, …]
variables:
  usuario: { default: ana }
  git_sha: { default: manual }
targets:
  dev:  { mode: development, default: true }
  qa:   { mode: production, root_path, run_as, permissions }
  prod: { mode: production, root_path, run_as, permissions }
```

Pieza	Para qué
bundle.name	el producto: nombra y aísla el despliegue
include	jobs, pipelines, apps… cada uno en su YAML
sync.exclude	lo que no viaja: material de clase, datos crudos, tests
variables	lo que cambia sin tocar código
targets	los ambientes y sus reglas

La regla de diseño: todo lo que varía entre ambientes es una variable o un ajuste del target. Si para ir a prod hay que editar un recurso, el bundle está mal diseñado.

Del diccionario de la clase 12 al YAML, y un test que los mantiene iguales

resources/job_diario.yml · tests/unit/test_bundle.py

Jobs API

```
# clase 12 · job.py
definicion_job(usuario, ruta_tareas, email)
  name: [xm-demanda] diario · ana
  notebook_path: <tu Git folder>/tareas/t1…
  email_notifications: ana@xm.com.co
api("POST", "create" | "reset")
```

bundle

```
# clase 13 · resources/job_diario.yml
name: … diario · ${var.usuario} · ${bundle.target}
parameters: ambiente = ${bundle.target}
notebook_path: ../notebooks/tareas/t1_bronce.py
on_failure: ${workspace.current_user.userName}
databricks bundle deploy -t <target>
```

Las tareas pasan a ser producto

De clases/c12-jobs/tareas a notebooks/tareas: material de clase y código desplegable no se mezclan. Todas leen ambiente y llaman a ubicacion(usuario, ambiente).

Dos descripciones del mismo job se separan solas

test_bundle.py compara YAML y job.py: mismo orden, dependencias, reintentos, run_if y el chequeo dependiendo de bronce. El CI lo detiene si divergen.

Variables y sustituciones: lo que cambia, declarado; lo que falta, detenido

Se resuelven al desplegar. Para pasar valores a una corrida, se usan parámetros del job

Prioridad	De dónde sale el valor de una variable
1	--var="usuario=ana" en el comando
2	variable de entorno BUNDLE_VAR_usuario (CI/CD)
3	.databricks/bundle/<target>/variable-overrides.json
4	variables: dentro del target
5	default de la variable

Sustitución	Se resuelve a
${var.usuario}	tu variable
${bundle.target}	dev · qa · prod
${workspace.current_user.userName}	quien despliega
${resources.jobs.x.id}	el id de otro recurso

El deploy que falla a propósito (lab 13, sección 4)

usuario no tiene default en el repo de la clase. El primer Deploy a dev se detiene en la validación, antes de tocar el workspace, y dice qué variable falta y dónde se puede poner. Ese error es una virtud: un deploy ambiguo no debe adivinar en qué esquema escribir. En CI/CD el mismo valor llega como BUNDLE_VAR_x, y git_sha llega del commit.

Dos modos: development para iterar, production para operar

mode: development en dev · mode: production en qa y prod

	mode: development	mode: production
Nombres	[dev <usuario>] [xm-demanda] diario · ana · dev	[xm-demanda] diario · ana · qa (tal cual)
Calendarios y disparadores	en pausa	como estén en el YAML
Desde el workspace	corre sobre tu Git folder (source-linked)	corre la copia sincronizada del deploy
Edición en la UI	permitida	bloqueada: se cambia en Git y se redespliega
Concurrencia y bloqueo	corridas concurrentes; sin bloqueo de deploy	con bloqueo: dos deploys no se pisan
Exige	nada	run_as y permissions explícitos, o un service principal; root_path fijo (una sola copia)

qa se configura igual que prod, siempre. Si qa difiere, lo que pasa en qa no dice nada de lo que pasará en prod.

Qué cambia entre ambientes, y qué no debe cambiar nunca

En una solución de mercado, esta frontera es la que permite responder a un auditor

Cambia (variable o target)

Dónde se escribe: esquema aquí; catálogo y workspace en Azure.
Con qué datos: dev y qa con copias; prod con la fuente oficial.
Quién ejecuta: tú en dev; un service principal en qa y prod.
Cada cuánto: en pausa en dev; 6:00 o al llegar el archivo en prod.
Quién se entera: tu correo; el grupo de operación.

No cambia nunca

El código de src/ y de las tareas.
El DAG y la política de fallas.
El contrato de oro y las reglas de calidad.
La versión del modelo que se promueve.
La definición de las métricas (WAPE).

Ejemplo de XM: si el pronóstico que recibe planeación se calcula con un código distinto del que se probó en qa, la pregunta "¿qué versión publicó este número, con qué datos, y quién la aprobó?" se rompe en el primer paso. ubicacion(usuario, ambiente) existe para que el código no sepa en qué ambiente está, salvo por un parámetro.

Free Edition y Azure: la misma idea, fronteras distintas

Aquí los ambientes son esquemas; en XM, workspaces y catálogos separados

Dónde vive cada ambiente: aquí, y en Azure Databricks

El código de las tareas es el mismo en las dos columnas: solo llama a ubicacion(usuario, ambiente).

Free Edition (este curso)

Un workspace · un metastore

catálogo workspace

`c01_ana` `c01_ana_qa` `c01_ana_prod`

`c01_luis` `c01_luis_qa` `c01_luis_prod`

Identidad: tú en los tres. Permisos: tus esquemas. Datos: qa y prod se siembran desde dev (tarea 0).

Azure Databricks (XM)

workspace dev

catálogo dev

`demanda_raw` `demanda_curated` `demanda_ml`

equipo de analítica · SP de dev

workspace qa

catálogo qa

`demanda_raw` `demanda_curated` `demanda_ml`

pipeline CI/CD · SP de qa

workspace prod

catálogo prod

`demanda_raw` `demanda_curated` `demanda_ml`

solo el SP de prod escribe · grupos leen

Un metastore por región · cada catálogo ligado solo a su workspace (workspace-catalog binding) · Key Vault y ADLS por ambiente

Lo que se simula aquí con esquemas, en Azure son fronteras reales: red, identidad, almacenamiento y facturación por ambiente.

Capa	Quién la crea	Con qué
Workspaces, catálogos, almacenamiento, redes, Key Vault, binding catálogo–workspace	Equipo de plataforma	Terraform (o el proveedor de la nube)
Jobs, pipelines, apps, modelos y su configuración por ambiente	Equipo de analítica	Bundle, desplegado por CI/CD

Terraform para la plataforma, bundles para el producto: analítica no necesita permisos para crear un workspace para desplegar su job.

Tres formas de desplegar el mismo bundle, con el mismo resultado

Hoy desde el workspace; en la clase 14, nadie despliega a mano a qa ni a prod

Desde el workspace

Abrir databricks.yml → ícono Deployments → elegir target → Deploy. Project output muestra la validación; Bundle resources, los jobs y el botón para correrlos. Sin instalar nada: ideal para Free Edition y para quien empieza.

Desde la CLI

databricks bundle validate -t qa
databricks bundle deploy -t qa
databricks bundle run job_diario -t qa
databricks bundle summary -t qa
En tu computador o en un runner, autenticado con databricks auth login.

Desde CI/CD

Los mismos comandos dentro de GitHub Actions o Azure DevOps, con la identidad del pipeline. Es la única forma aceptable para qa y prod: deja registro de qué commit, cuándo y quién aprobó.

Lo importante no es el botón: es que los tres leen el mismo databricks.yml. Si un deploy desde la UI y uno desde CI producen jobs distintos, el problema está en el repo, no en la herramienta.

Un ambiente nuevo se prepara con un job, no a mano

resources/job_preparar_ambiente.yml · notebooks/tareas/t0_preparar_ambiente.py · idempotente

1

Esquema y volumen

CREATE SCHEMA / VOLUME IF NOT EXISTS c01_<usuario>_qa

2

Archivos de landing

Se copian los que falten desde dev; nunca se sobrescribe uno que ya está

3

Tabla de referencia

dim_ciiu con DEEP CLONE: datos y metadatos, sin depender de dev

4

Modelo

copy_model_version de @champion y @challenger, con tag origen

qa no lee dev

Construye sus propias tablas desde su propio landing. Lo único que viene de dev es lo que esta tarea copió, de forma explícita y con rastro.

Promover el artefacto: ¿está bien?

Rápido y trazable, pero un modelo entrenado con datos de dev llega a prod. La clase 14 lo discute (deploy code, not models) y la 15 reentrena en el ambiente.

Lab 13: el mismo código en dev y qa, desplegado con un bundle

clases/c13-bundles/notebooks/lab13_bundles_ambientes · widget usuario · rama feature/c13-bundles

1

TODO 1 · esquema_de()

utils/ambiente.py: c01_<usuario><sufijo>; un ambiente no se adivina

2

Deploy a dev, el que falla · TODO 2

Panel Deployments → dev → falla por usuario → default en databricks.yml → Deploy

3

Correr el job de dev

Sin datos nuevos: la rama se excluye, el chequeo corre

4

TODO 3 · target qa

Igual a prod: modo producción, root_path, run_as, permissions → Deploy a qa

5

preparar_ambiente y job_diario en qa

qa reproduce el pronóstico de dev con sus propias tablas

Criterios de aceptación

- test_ambiente y test_bundle sin skip
- Job de dev desplegado por el bundle
- c01_<usuario>_qa poblado por su job
- Modelo de qa con tag origen
- Dos preguntas · verificar() en OK · PR en verde

No despliegues prod

Lo hace el pipeline de la clase 14, después de qa y con una aprobación. Cola de Free Edition: 5 tareas por cuenta.

Duración: 50 min con el notebook guiado